```
    volatile atomic<BaseType>* p,BaseType * old_value,
    BaseType new_value, memory_order success_order,
    memory_order failure_order) noexcept;
template<typename BaseType>
bool atomic_compare_exchange_weak_explicit(
    atomic<BaseType>* p,BaseType * old_value,
    BaseType new_value, memory_order success_order,
    memory_order failure_order) noexcept;
```

Effects

```
return p->compare_exchange_weak(
    *old_value,new_value,success_order,failure_order);
```

D.3.9. Specializations of the std::atomic template

Specializations of the `std::atomic` class template are provided for the integral types and pointer types. For the integral types, these specializations provide atomic addition, subtraction, and bitwise operations in addition to the operations provided by the primary template. For pointer types, the specializations provide atomic pointer arithmetic in addition to the operations provided by the primary template.

Specializations are provided for the following integral types:

```
std::atomic<bool>
std::atomic<char>
std::atomic<signed char>
std::atomic<unsigned char>
std::atomic<short>
std::atomic<unsigned short>
std::atomic<int>
std::atomic<unsigned>
std::atomic<long>
std::atomic<unsigned long>
std::atomic<long long>
std::atomic<unsigned long long>
std::atomic<wchar_t>
std::atomic<char16_t>
std::atomic<char32_t>
```

and for `std::atomic<T*>` for all types `T`.